

AXI4-LITE SLAVE INTERFACE

Prepared by

Ahmed Osama Mohamed

Fayza Mohsen Farouk

Noura Allah Ahmed Hussein

AMPA Bus
Protocol

Table of Content

1. Project Overview
2. Requirements and Specifications
3. Design Decisions and System Architecture
4. AXI4-Lite Interface
5. Write Controller
6. Read Controller
7. Register File
8. UART Interface
9. Reset and Clocking
10. Error and Status Handling
11. Verification and Testbench
12. Synthesis and Implementation
13. Conclusion

1. Project Overview

1.1 Project Description

This project implements an **AXI4-Lite Slave Interface** using Verilog HDL to handle read and write transactions between an AXI4-Lite Master and an internal Register File. The design also includes a **UART interface** for serial data transmission and reception.

1.2 Project Objectives

- Implement AXI4-Lite read/write transactions.
- Design Read and Write Controllers using FSMs.
- Implement register access and address decoding.
- Handle errors and read timeouts.
- Verify the design using a testbench and simulation.

1.3 Main Features

- AXI4-Lite **VALID/READY handshake**.
- Separate **Read and Write Controllers**.
- Address-based Register File access.
- **WSTRB** support.
- Error and timeout handling.
- Parameterized address/data widths.
- Verilog simulation and verification.
- UART serial communication.
- UART TX and RX support.

2. Requirements and Specifications

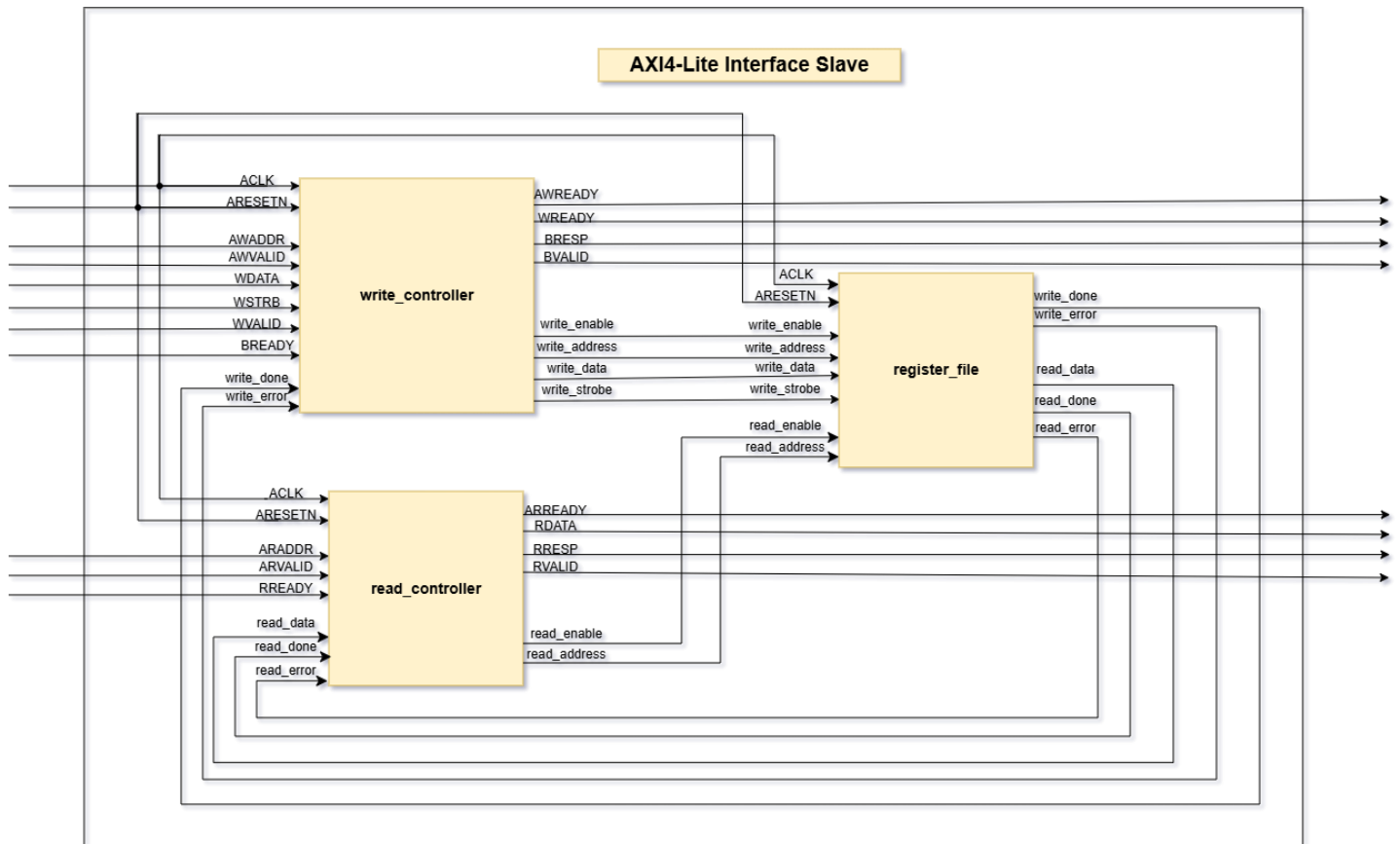
The project aims to implement a configurable AXI4-Lite slave for accessing a memory-mapped register file.

The design shall support both read and write transactions through dedicated Read and Write Controllers. These controllers are responsible for managing the transaction flow between the AXI4-Lite interface and the Register File.

The Register File shall provide registers with different access permissions and handle the actual register access. The design shall also support byte-level write operations, detect invalid accesses, and provide status information for successful or failed transactions.

The system shall maintain counters for read/write operations and their corresponding errors. The design shall be parameterized to allow different data and address widths.

3. Design Decisions and System Architecture



The design consists of four main functional blocks: the Write Controller, Read Controller, Register File, and UART interface. The Write and Read Controllers manage AXI4-Lite transactions, the Register File provides memory-mapped register storage and access control, and the UART interface handles serial data transmission and reception.

The Register File is responsible for register storage, address validation, access permission checking, and read/write operations.

4. AXI4-Lite Interface

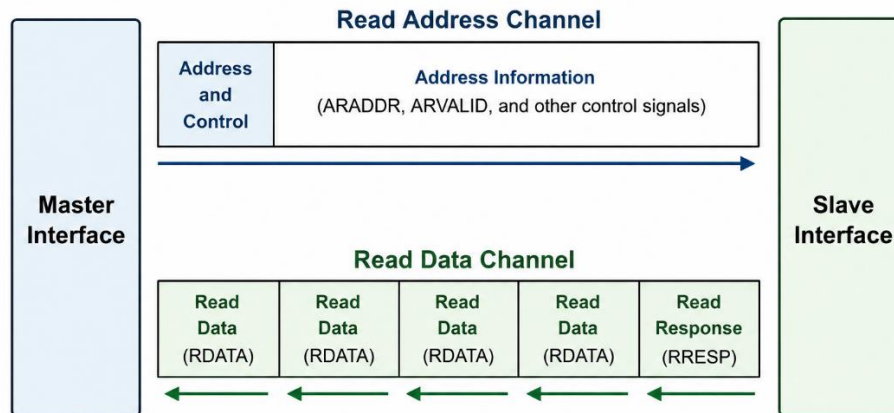
4.1 AXI4-Lite Overview

AXI4-Lite is a memory-mapped communication protocol used to transfer data between the Master and the Register File. In this project, it is used to perform read and write operations.

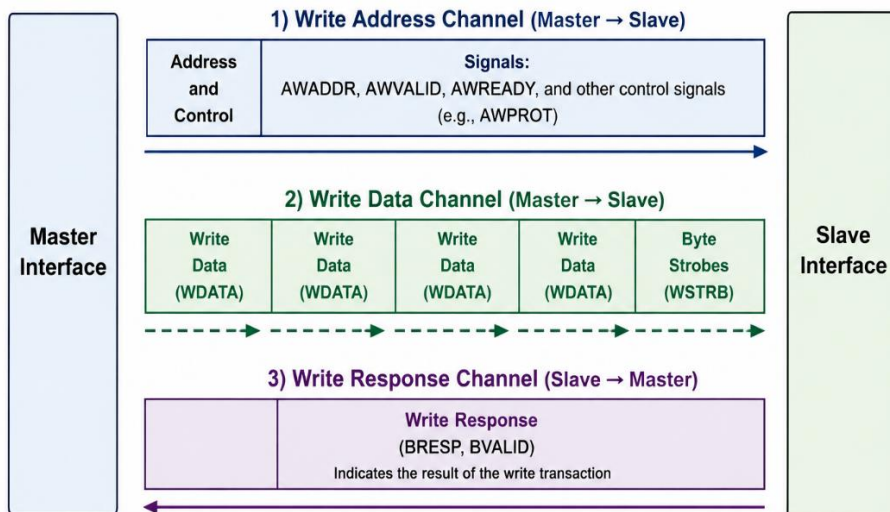
4.2 Addressing

The Register File uses memory-mapped addressing. Each register has a specific address, and the received address is decoded to select the required register for read or write operations.

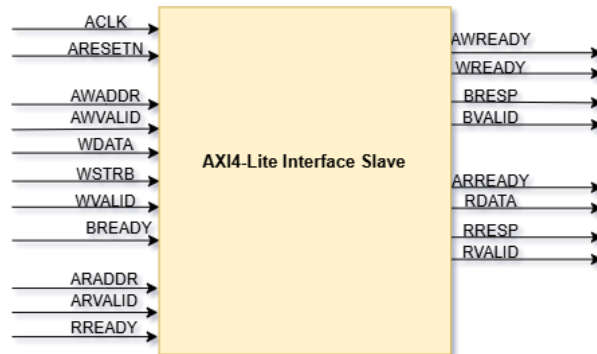
✓ AXI4-Lite Read Address and Read Data Channels



✓ AXI4-Lite Write Address, Write Data, and Write Response Channels



4.3 AXI Signals



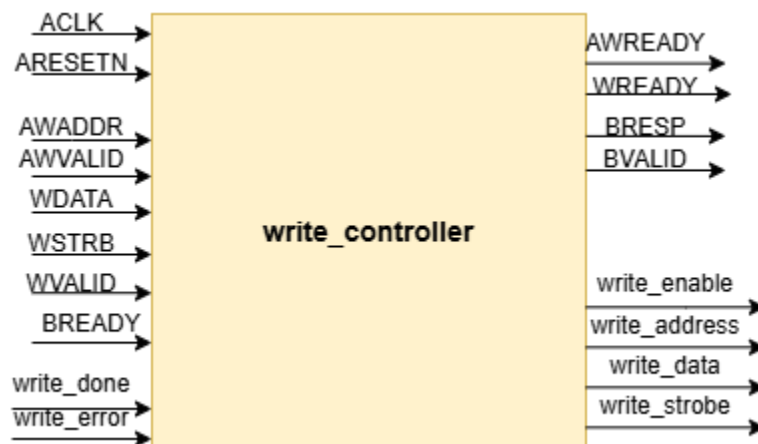
AXI4-Lite signals		Description
ARADDR		Read address.
ARVALID		Indicates a valid read address.
ARREADY		Indicates that the slave is ready to receive the address
RDATA		Read data.
RVALID		Indicates that read data is valid
RREADY		Indicates that the Master is ready to receive the data.
RRESP		Read response.
AWADDR		Write address.
WDATA		Write data.
WSTRB.		Selects which bytes are written
BVALID		Indicates a valid write response.
BRESP		Write response.

5. Write Controller

5.1 Purpose and Responsibilities

The Write Controller manages the AXI4-Lite write transaction. It receives the write address and data from the Master, requests the write operation from the Register File, and returns a success or error response to the Master.

5.2 Inputs and Outputs



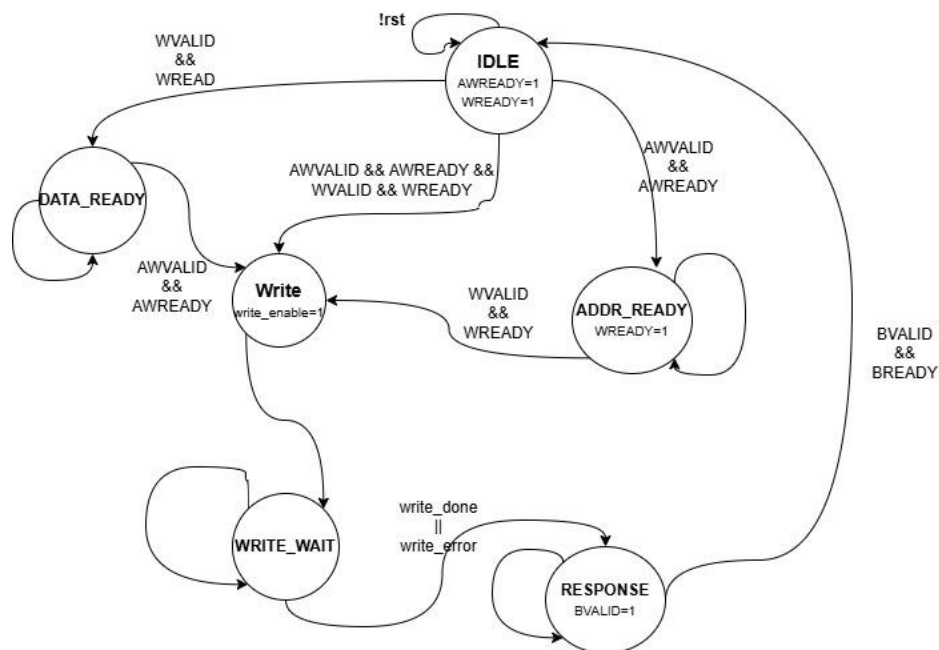
Signal	Description
ACLK	System clock.
ARESETN	Active-low asynchronous reset.
AWADDR	Write address received from the Master.
AWVALID	Indicates that a valid write address is available.
AWREADY	Indicates that the controller is ready to accept the write address.
WDATA	Write data received from the Master.
WSTRB	Indicates which bytes of WDATA are valid for writing.
WVALID	Indicates that valid write data is available.
WREADY	Indicates that the controller is ready to accept the write data.
BRESP	Write response indicating success or error.
BVALID	Indicates that the write response is valid.
BREADY	Indicates that the Master is ready to accept the write response.
write_enable	Requests a write operation from the Register File.
write_address	Write address sent to the Register File.
write_data	Write data sent to the Register File.
write_strobe	Byte-write information sent to the Register File.
write_done	Indicates that the Register File completed the write successfully.
write_error	Indicates that the Register File detected a write error.

5.3 Write Transaction Flow

The controller:

- 1) **AWREADY** is asserted to receive the write address. The address is accepted when **AWVALID** and **AWREADY** are both asserted.
- 2) **WREADY** is asserted to receive the write data. The data is accepted when **WVALID** and **WREADY** are both asserted.
- 3) **AWADDR** is stored in **awaddr_reg**, while **WDATA** and **WSTRB** are stored in **wdata_reg** and **wstrb_reg**.
- 4) **write_enable** is asserted, and the stored address, data, and strobe are sent to the Register File through **write_address**, **write_data**, and **write_strobe**.
- 5) The controller waits for **write_done** or **write_error** from the Register File.
- 6) **BRESP** is set to 2'b00 for a successful write and 2'b10 for a write error.
- 7) **BVALID** is asserted to indicate that the response is available and remains asserted until **BVALID** and **BREADY** are both asserted.

5.4 FSM Architecture



- **IDLE** — Initial state; accepts the write address and/or write data.
- **ADDR_READY** — Waits for the write data after receiving the address.
- **DATA_READY** — Waits for the write address after receiving the data.
- **WRITE** — Initiates the write operation to the Register File.
- **WRITE_WAIT** — Waits for the write operation result.
- **RESPONSE** — Sends the write response to the Master.

5.5 State Transitions

- 1) **IDLE** → **WRITE**: Address and data are accepted simultaneously.
- 2) **IDLE** → **ADDR_READY**: Address is accepted first.
- 3) **IDLE** → **DATA_READY**: Data is accepted first.
- 4) **ADDR_READY** → **WRITE**: Write data is received.
- 5) **DATA_READY** → **WRITE**: Write address is received.
- 6) **WRITE** → **WRITE_WAIT**: Write request is issued to the Register File.
- 7) **WRITE_WAIT** → **RESPONSE**: **write_done** or **write_error** is received.
- 8) **RESPONSE** → **IDLE**: **BVALID** and **BREADY** handshake is completed.

5.6 Write Timing

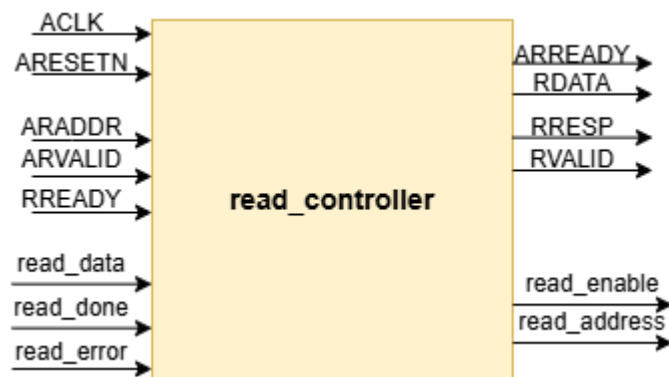
The Write Controller operates synchronously with **ACLK**. After the write address and data are accepted, the write operation is initiated and the controller waits for **write_done** or **write_error**. Once a response is received, **BVALID** is asserted and remains active until the **BVALID–BREADY** handshake is completed.

6. Read Controller

6.1 Purpose and Responsibilities

The Read Controller manages the AXI4-Lite read transaction. It receives the read address from the Master, requests the corresponding data from the Register File, and returns the data or an error response to the Master.

6.2 Inputs and Outputs



Signals of the Read Controller	Description
ARADDR.	Read address received from the Master
ARVALID	Indicates that the read address is valid.
ARREADY	Indicates that the controller is ready to receive the address.
read_enable	Requests a read operation from the Register File.
read_address	Address sent to the Register File.
read_data	Data received from the Register File.
read_done	Indicates that the requested data is available.
read_error	Indicates a read error.
RDATA	Read data sent to the Master.
RRESP	Read response status.
RVALID	Indicates that the read response is valid.
RREADY	Indicates that the Master is ready to receive the response.

6.3 Read Transaction Flow

The read operation follows these steps:

- 1) The Master sends **ARADDR** with **ARVALID** = 1.
- 2) The controller accepts the address when **ARREADY** = 1.
- 3) The controller enters the **READ** state and asserts **read_enable**.
- 4) The Register File returns **read_data** with **read_done**.
- 5) If an error occurs or timeout counter reaches 30 cycles, an error response is sent.
- 6) The controller enters the **RESP** state and asserts **RVALID**.
- 7) The transaction finishes when **RVALID** = 1 and **RREADY** = 1.

6.4 FSM Architecture

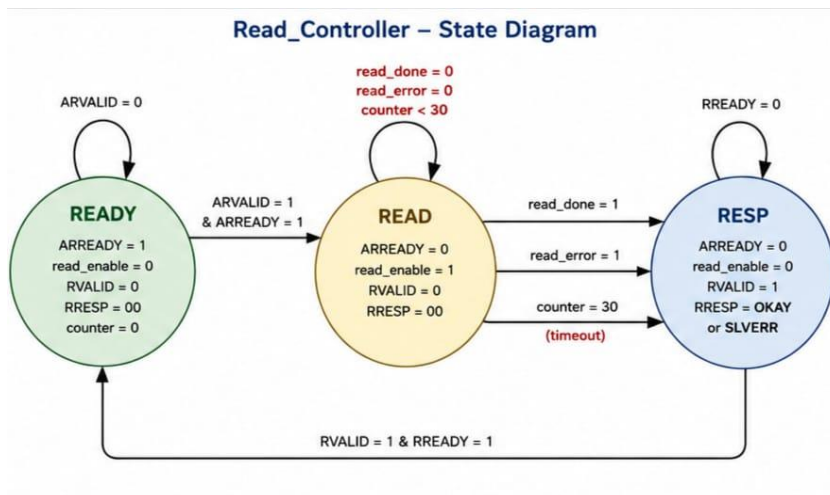
The Read Controller is implemented using a Finite State Machine (FSM) with three states:

- **READY:** Waits for a valid read address from the Master.
- **READ:** Requests data from Register File and waits for **read_done**, **read_error**, or timeout.
- **RESP:** Sends the read data and response to the Master.

6.5 State Transitions:

The state transitions are:

- 1) **READY → READ:** When **ARVALID**=1 and **ARREADY**=1.
- 2) **READ → RESP:** When **read_done**=1, **read_error**=1, or the counter reaches 30 cycles.
- 3) **RESP → READY:** When **RVALID**=1 and **RREADY**=1.



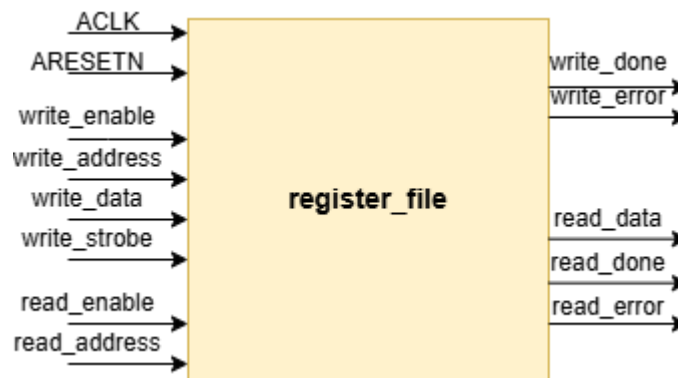
6.6 Read Timing

The Read Controller operates synchronously with **ACLK**. After accepting the read address, the controller enters the **READ** state and asserts **read_enable**. The counter starts counting clock cycles while waiting for the Register File response.

If **read_done** or **read_error** occurs, the controller moves to the **RESP** state. If no response is received within 30 clock cycles, a timeout error is generated.

7. Register File

7.1 Register File Architecture



The storage and register-access block of the design. It contains the register array, access-type information, and counters for read and write operations.

It receives address and control information from the Read and Write Controllers, validates the requested operation, and performs the required register access.

Some registers are assigned to control and monitor the UART interface, providing a mapped interface between the AXI4-Lite Master and UART functionality.

The main components of the Register File are:

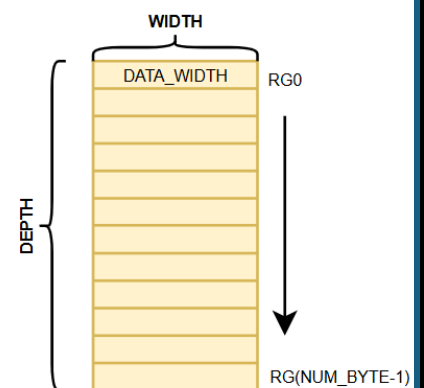
- Register Array: Stores the actual register values.
- Register Type Array: Defines the access permission of each register.
- Address Decoder: Maps the byte address to the corresponding register index.
- Counters: Store the number of successful and erroneous read/write operations.

7.2 Register Array

The Register Array represents the actual storage space of the Register File.

Each register has a fixed width determined by `DATA_WIDTH`, while the number of registers is determined by `ADDR_WIDTH` and the number of bytes per register. The registers are initialized to zero during reset.

Register contents are updated synchronously with the system clock during valid write operations.



7.3 Register Addressing and Access Control

The Register File uses the received byte address to identify the target register. The address is checked for proper alignment, and the selected register's access type is checked to determine whether the operation is permitted. An operation is valid only when both conditions are satisfied.

7.4 Read and Write Operations

A read or write operation is initiated when the corresponding enable signal is asserted. A valid read returns the selected register value, while a valid write updates the selected register according to the write strobe.

7.5 Statistics Counters

The Register File includes four counters to monitor successful and rejected read/write operations. The AXI Master can read these counters to monitor register activity and access errors.

Register Location	Counter	Access Type	Function
Last Register	Write Counter	R	Counts successful write operations
Second-to-Last Register	Read Counter	R	Counts successful read operations
Third-to-Last Register	Write Error Counter	R	Counts rejected write operations
Fourth-to-Last Register	Read Error Counter	R	Counts rejected read operations

7.6 Write Strobe

The Write Strobe provides byte-level control during write operations. Each bit of the strobe corresponds to one byte of the [DATA_WIDTH](#) register, allowing specific bytes to be updated while preserving the contents of the remaining bytes.

8. UART Interface

8.1 UART Overview

The UART interface provides serial communication through separate **Transmitter (TX)** and **Receiver (RX)** modules. It supports parameterized data width and clock cycles per bit.

8.2 UART Transmitter

The UART Transmitter converts parallel `tx_data` into a serial bit stream. It uses four FSM states: IDLE, START, DATA, and STOP. The transmitter sends each byte sequentially, starting with a start bit, followed by 8 data bits and a stop bit.

8.3 UART Receiver

The UART Receiver converts the incoming serial `rx` signal into parallel `rx_data`. It detects the start bit, samples the 8 data bits, and processes each byte sequentially until the complete data word is received. `rx_valid` is asserted when the complete data word is available.

8.4 UART Signals

Signal	Description
ACLK	System clock.
ARESETN	Active-low asynchronous reset.
rx	Serial data input to the Receiver.
rx_data	Received parallel data.
rx_valid	Indicates that a complete data word has been received.
tx_data	Parallel data to be transmitted.
tx_start	Starts the transmission.
tx_busy	Indicates that the Transmitter is busy.
tx	Serial data output from the Transmitter.

8.5 UART FSM Architecture

Both the Transmitter and Receiver use the same four-state FSM structure:

- **IDLE** — Waits for a transmission request or detects the start of incoming data.
- **START** — Generates or processes the UART start bit.
- **DATA** — Transmits or receives the 8 data bits of each byte.
- **STOP** — Generates or processes the stop bit before moving to the next byte or completing the transfer.

8.6 UART Data Handling

The UART supports multi-byte data using `byte_counter`. Each byte is processed separately, with `bit_counter` selecting the individual bits within the byte. The `baud_counter` controls the timing of bit transmission and sampling according to `CLKS_PER_BIT`.

8.7 UART Timing

The UART timing is controlled by `baud_counter`. The Transmitter changes or maintains each bit for `CLKS_PER_BIT` clock cycles. The Receiver samples the start bit at approximately half a bit period and samples the data bits according to `CLKS_PER_BIT`.

9. Reset and Clocking

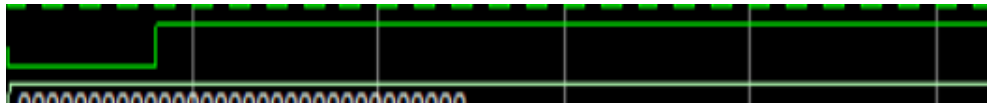
9.1 Clocking Scheme

ACLK is the main system clock. In the testbench, it is generated using **always #5 ACLK = ~ACLK**; resulting in a **10 ns period (100 MHz)**. AXI4-Lite transactions are synchronized with the clock edges.



9.2 Reset Scheme

ARESETN is an **active-low reset**. The testbench holds the design in reset for **three clock cycles** before starting normal operation.



9.3 Sequential Logic

AXI4-Lite read and write transactions are synchronized with the rising edge of **ACLK**. The testbench waits for posedge **ACLK** to detect the **ARVALID/ARREADY**, **AWVALID/AWREADY**, and **WVALID/WREADY** handshakes, as well as the **RVALID/RREADY** and **BVALID/BREADY** response handshakes. This ensures that read and write operations are performed synchronously with the system clock.

9.4 Reset Behavior

During reset, AXI control signals, addresses, data, and test variables are initialized to **known values**. After reset is released, normal transactions begin.

10. Error and Status Handling

10.1 Write Status

The `write_done` signal indicates that a write transaction has been successfully completed.

10.2 Read Status

The `read_done` signal indicates that a read transaction has been successfully completed and the requested data is available.

10.3 Error Conditions

The design detects invalid transactions, including:

- Misaligned addresses.
- Write access to read-only registers.
- Read access to write-only registers.

The corresponding `write_error` or `read_error` signal is asserted when an invalid transaction occurs.

10.4 Error Counters

Two counters are used to monitor failed transactions:

- `write_error_counter`: counts invalid write transactions.
- `read_error_counter`: counts invalid read transactions.

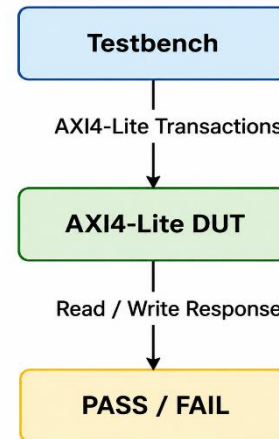
10.5 Status Signaling

The done and error signals provide the transaction result to the corresponding controller. The controller uses this result to generate the appropriate response to the AXI4-Lite Master.

11. Verification and Testbench

11.1 Testbench Architecture

The testbench generates the clock and reset signals, drives AXI4-Lite read and write transactions, monitors the DUT responses, and checks the expected results.

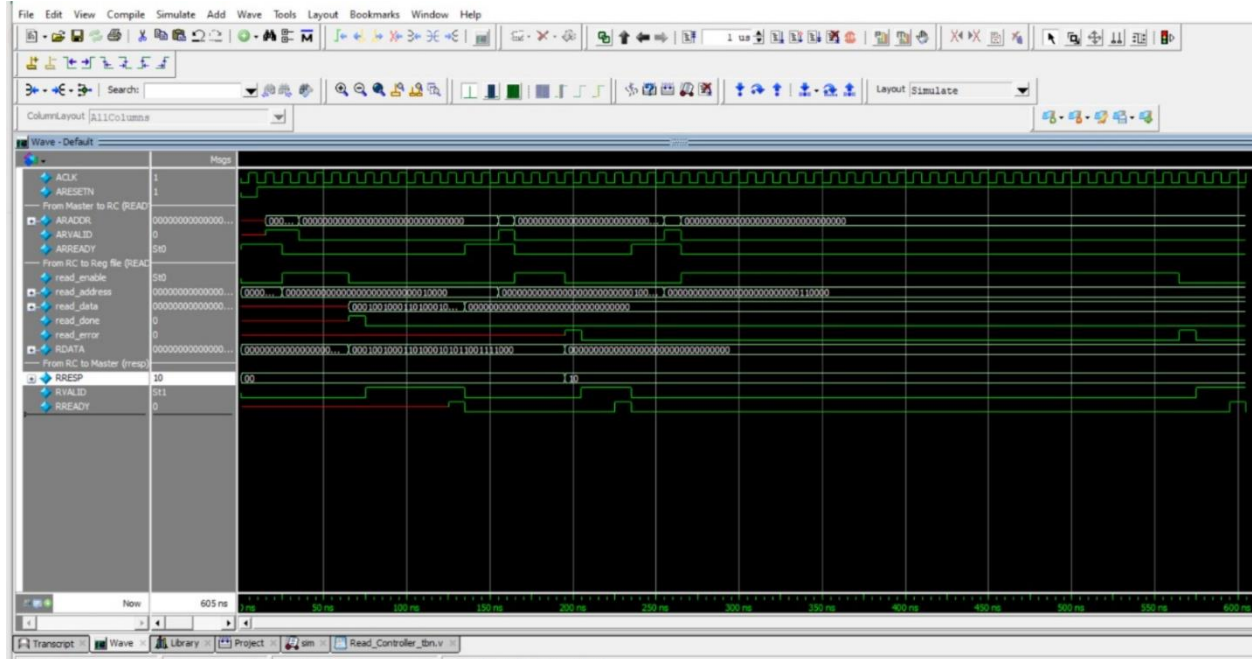


11.2 Valid Write Tests

A valid write transaction was performed by sending a valid address and data. The expected response was $BRESP = 2'b00$ (OKAY).

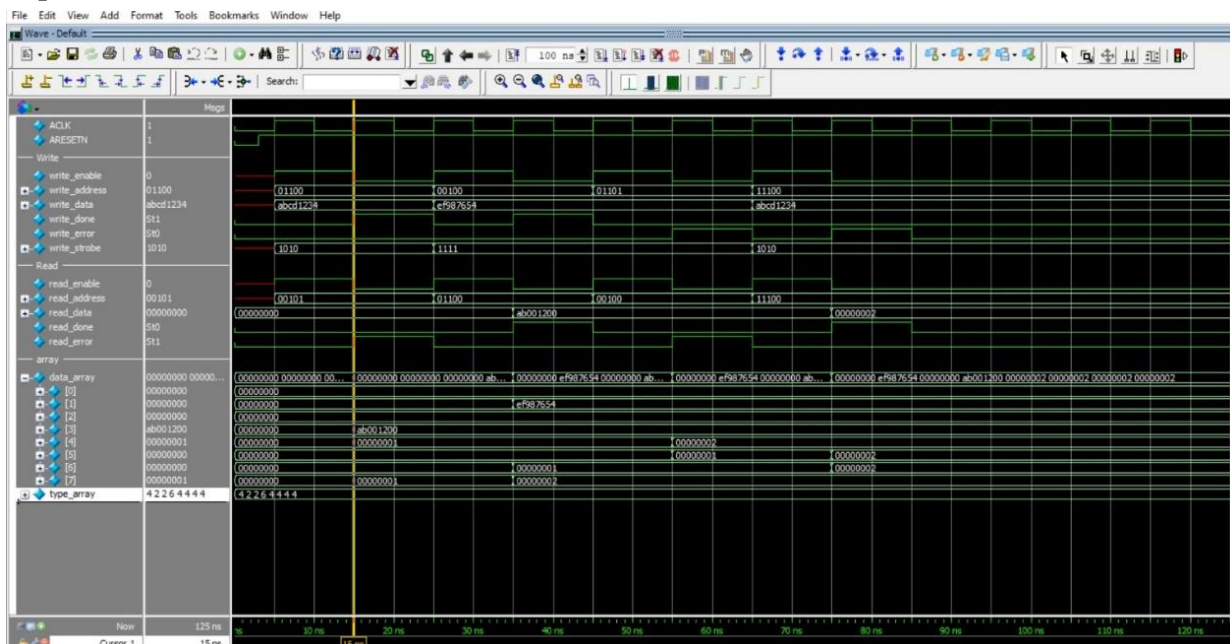
11.3 Valid Read Tests

A valid read transaction was performed after writing data to a register. The expected data was returned with RRESP = 2'b00 (OKAY).



11.4 Register File Verification

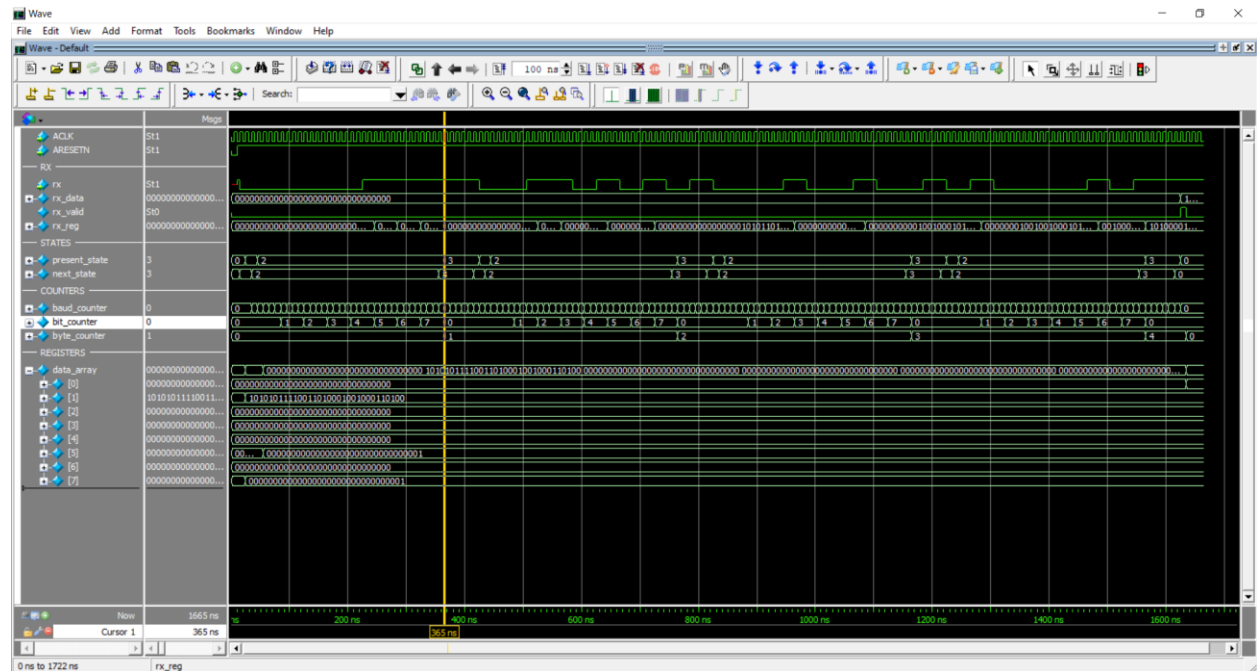
The Register File was verified to ensure that data is correctly stored and retrieved from the specified registers. The testbench checks that the returned register value matches the expected value.



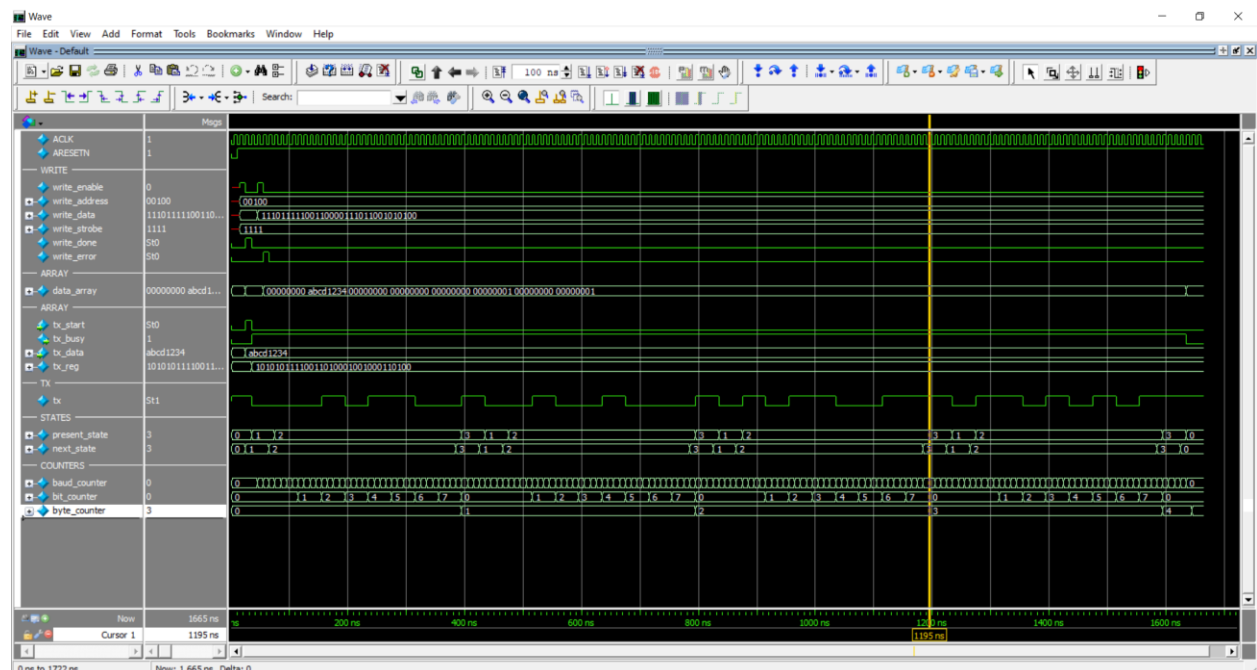
11.5 UART Verification

The UART interface was verified by testing serial data transmission and reception. The testbench checks that transmitted data is correctly serialized and that received serial data is correctly reconstructed into the expected parallel data.

UART RX

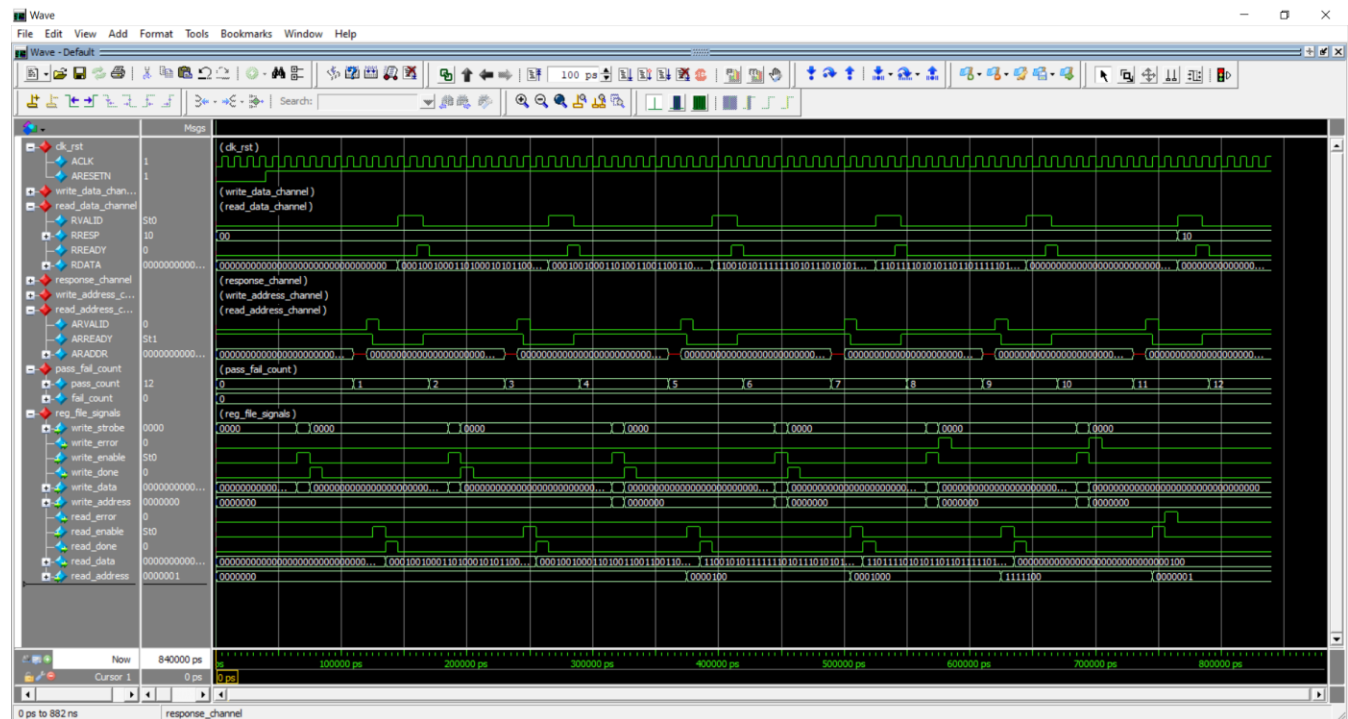
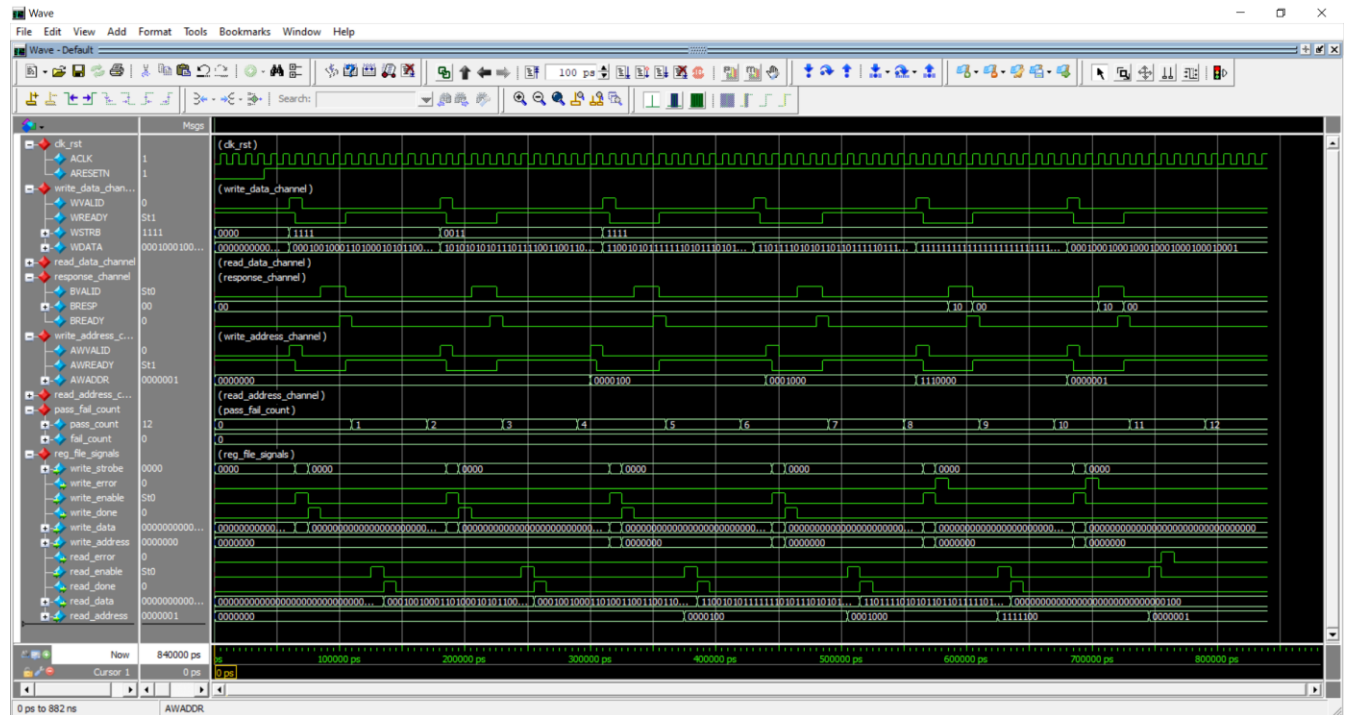


UART TX



11.6 Simulation Results

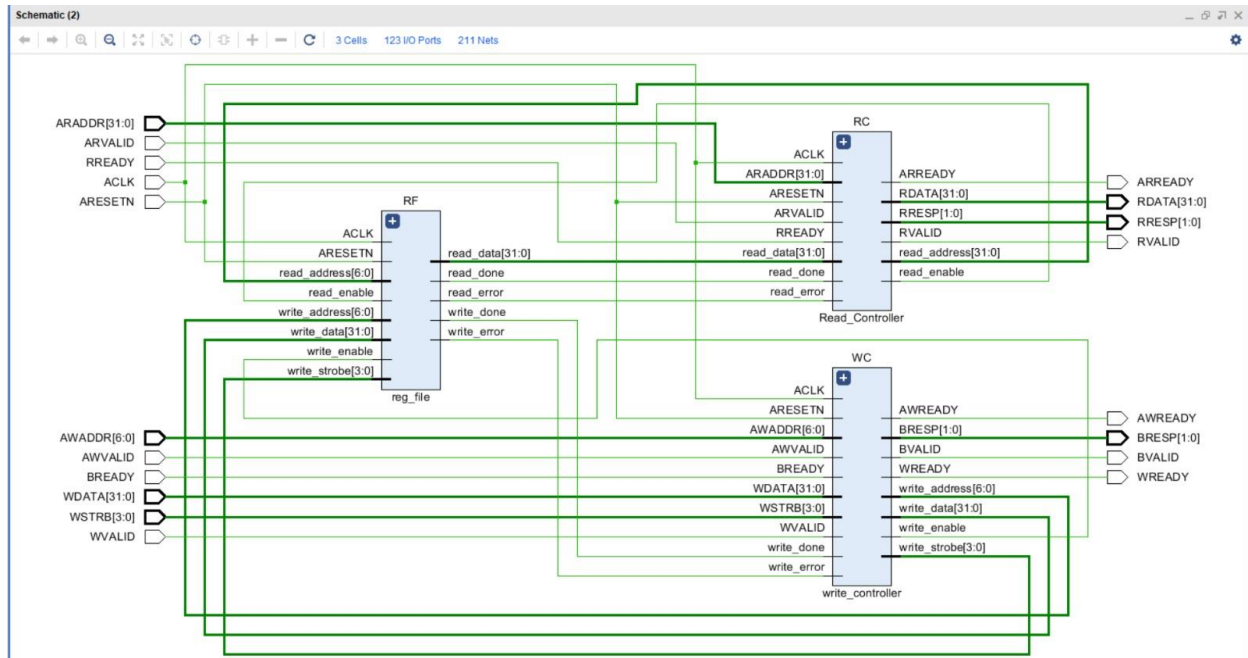
The simulation results confirmed the correct operation of the implemented AXI4-Lite transactions. The testbench tracks the number of passed and failed tests to verify the overall functionality.



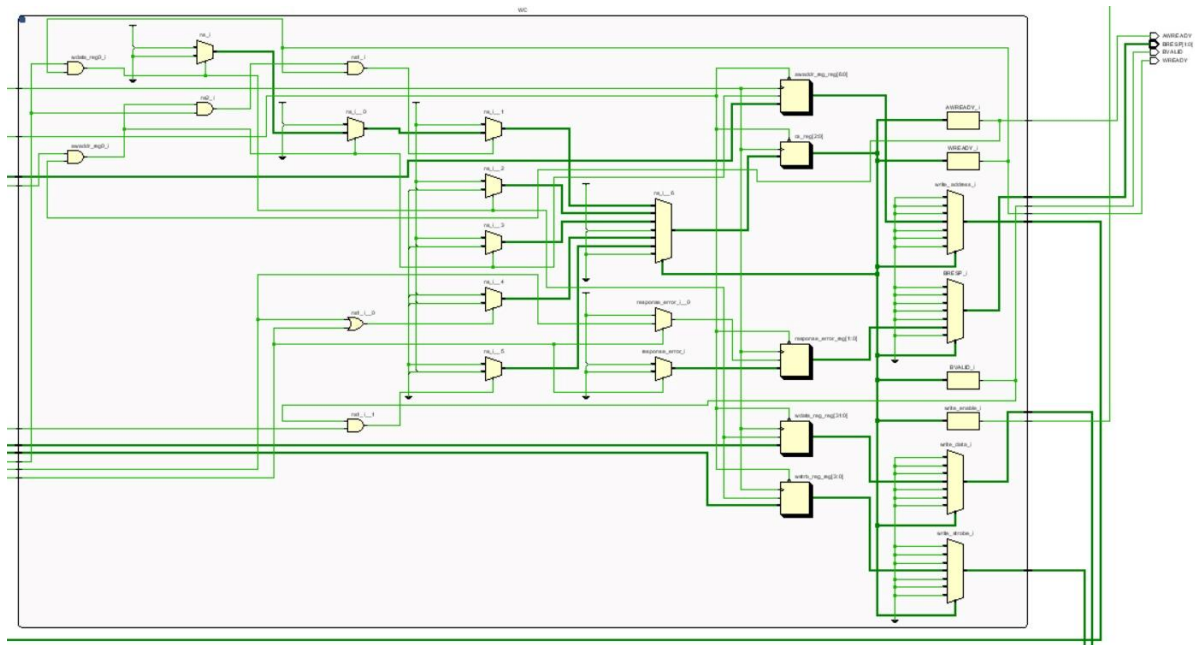
12. Synthesis and Implementation

12.1 Overall System RTL Schematic

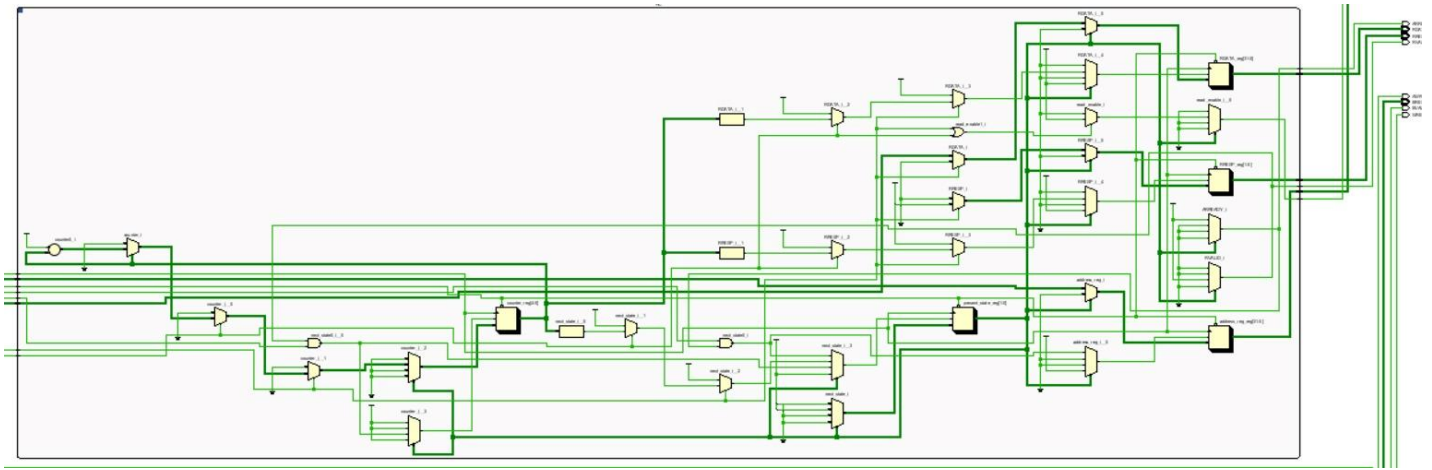
The design was implemented in Xilinx Vivado to verify the synthesized hardware structure and RTL connectivity.



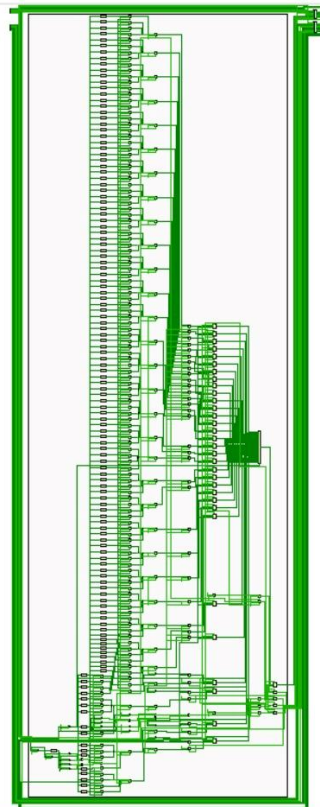
12.2 Write Controller RTL Schematic



12.3 Read Controller RTL Schematic



12.4 Register File RTL Schematic



13. Conclusion

The project successfully implemented an **AXI4-Lite interface** with separate **Write Controller, Read Controller, and Register File** blocks. The design supports register access, address alignment, access permissions, WSTRB, reset handling, and error responses. The testbench was used to verify the main read and write operations and confirm the correct behavior of the design through simulation. The design also includes **UART** serial communication, providing data transmission and reception capabilities alongside the AXI4-Lite register interface.

As a next step, the design is implemented and tested using **Xilinx Vivado** to verify its synthesis and hardware implementation, in addition to the functional simulation performed using **ModelSim**.